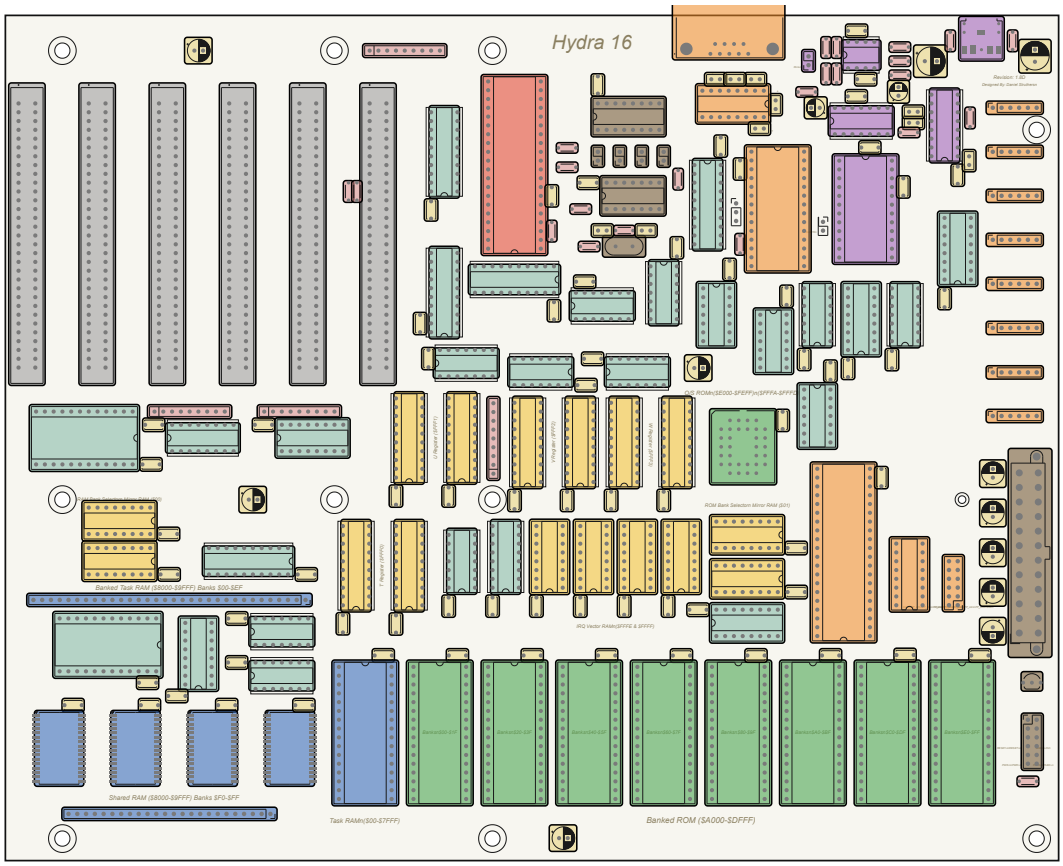
THE HYDRA PROJECT

HYDRA-16

Technical Reference



Every keyword, word, form, command and system call:
its syntax, its exact behaviour, its limits and its errors

HydraOS 1.0 · preview

About This Reference

This is a **preview** of the *Hydra-16 Technical Reference*: ten entries, at least one from each kind of thing the finished book will cover, written to settle its depth and its tone before the rest is done.

The *User's and Builder's Manual* teaches the Hydra, and its appendices list what each language has in a line apiece. This book is where to look when a line isn't enough: when the question is what a keyword does with an empty string, which errors a call can return, or what is left in a variable after a loop ends. It doesn't teach. It assumes the manual has been read, and it says the same thing about every entry, in the same order.

What it will hold

CHAPTER	ENTRIES	IN THIS PREVIEW
1. HyBASIC keywords	Every statement, function and operator	`ABS`, `ASC`, `FOR...NEXT`, `INPUT`, `OPEN`
2. Commands	The shell's own commands, and the programs in `/bin`	`cp`
3. System calls	Every call a program can make	`OPEN`, `READ`
4. HyForth words	The core and every library	`>number`
5. hylang forms	The special forms and the functions	`let`

The other reference appendices of the manual (the 65C02's instructions, the character sets, the memory map, the errors) will follow in the same way.

How to read an entry

Every entry has the same parts, in this order. A part with nothing to say is left out.

`Syntax`
: Each form the thing can take. Words in `CAPITALS` (or, for a command or a HyForth word, as printed) are typed as they are; *italic* names stand for what you supply; `[ ]` encloses what can be left out; `{ | }` is a choice of one; `...` means the last item can repeat.

`Arguments`
: Each one: its type, its range, and what it means. For a system call, the registers in.

`Result`
: What a function returns, a word leaves on the stack, or a call leaves in the registers.

`Description`
: What happens, in the order it happens: what is evaluated and when, what is changed, where it stops.

`Errors`
: Each error it can raise, with its number and its message as the Hydra prints it, and what causes it.

Notes
: Limits, edge cases, and behaviour that differs from what another system's user would expect.

See also
: Related entries.

Example
: A transcript, taken from the Hydra. Every one in this book is replayed in the emulator when the book is built, so what is printed here is what the ROM does.

How it was written

Each entry was written from the interpreter's, the program's or the kernel's source, and each statement of behaviour in it was then tried on the system. Where the source and the trial disagreed with what one would expect of the language elsewhere (QuickBASIC, the Forth standard), the entry says what the Hydra does.

Section and chapter numbers such as "section 13.2" and "Appendix G" are the *User's and Builder's Manual*'s.

Contents

About This Reference **2**

1 HyBASIC Keywords **5**
ABS 5
ASC 6
FOR ... NEXT 7
INPUT 8
OPEN 11

2 Commands **13**
cp 13

3 System Calls **16**
OPEN 16
READ 18

4 HyForth Words **21**
>number 21

5 hylang Forms **23**
let 23

CHAPTER 1

HyBASIC Keywords

HyBASIC compiles a program whole before it runs it: every line is read, every name resolved and every block matched, and only then does the first statement execute. An error the compiler finds (a `FOR` with no `NEXT`, a label that isn't there) is therefore reported before anything has run, with the line it was found on. A line typed at the prompt without a number is compiled and run the same way, as a program of one line.

An error is reported as its message, in small letters. `ERR` then holds its number, `ERR$` the message and `ERL` the line; the numbers given in each entry are `ERR`'s. A failure reported by the system (a disk that is full, a name that isn't there) has HyBASIC's own number where there is one that fits (53, file not found), and otherwise 256 plus the system's code (Appendix G).

A number is an integer of any length, a rational, a decimal or a complex number, and arithmetic on them is exact (Chapter 18). A string is a sequence of bytes, each 0 to 255, up to 64K of them.

ABS

A numeric function: the absolute value of a real number.

SYNTAX

```
ABS(x)
```

ARGUMENTS

x

A numeric expression whose value is real: an integer, a rational or a decimal.

RESULT

x if *x* is zero or more, and *–x* if it is less. The result is the same kind of number as *x*, and exact: an integer gives an integer, of whatever length; a rational gives a rational in lowest terms; a decimal gives a decimal with the same digits. There is no largest value and no overflow.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*x* is complex. A complex number has no sign, and `ABS` does not give its modulus
13	`type mismatch`	*x* is a string
2	`syntax error`	No argument, or more than one

NOTES

For the modulus of a complex number *z*, use `SQR(REAL(z) ^ 2 + IMAG(z) ^ 2)`.

SEE ALSO

`SGN`, `INT`, `FIX`, `REAL`, `IMAG`.

EXAMPLE

```
> PRINT ABS(-7); ABS(7); ABS(-7/2); ABS(-2.5)
 7  7  7/2  2.5
> PRINT ABS(-2 ^ 100)
 1267650600228229401496703205376
> PRINT ABS(3+4i)
illegal function call
> PRINT ERR; SQR(REAL(3+4i) ^ 2 + IMAG(3+4i) ^ 2)
 5  5
```

ASC

A numeric function: the code of a string's first byte.

SYNTAX

```
ASC(s$)
```

ARGUMENTS

s$

A string expression of one byte or more.

RESULT

An integer from 0 to 255: the value of the first byte of *s$*. The rest of the string is not looked at. A string holds bytes, not characters of any one set, so the result is the byte itself; which character the console draws for it depends on the character set in use (Appendix E). The codes 32 to 126 are ASCII's.

ERRORS

ERR	MESSAGE	CAUSE
5	`illegal function call`	*s$* is the empty string
13	`type mismatch`	The argument is a number

NOTES

`ASC` is the inverse of `CHR$` for one byte: `ASC(CHR$(n))` is *n* for every *n* from 0 to 255, including 0 (a zero byte doesn't end a string). To test a string that may be empty, test `LEN(s$)` first.

SEE ALSO

`CHR$`, `LEN`, `MID$`, `VAL`.

EXAMPLE

```
> PRINT ASC("A"); ASC("Abc"); ASC(" "); ASC(CHR$(0) + "x"); ASC(CHR$(255))
 65  65  32  0  255
> PRINT ASC("")
illegal function call
```

FOR ... NEXT

A statement block: a loop that counts a variable from one value to another.

SYNTAX

```
FOR v = start TO limit [STEP step]
    [statements]
NEXT [v [, v] ...]
```

ARGUMENTS

v

The loop's counter: a simple numeric variable. An array's element can't be one.

start, limit, step

Numeric expressions. *step* is 1 if `STEP` is left out.

DESCRIPTION

1. *start* is assigned to *v*, and *limit* and *step* are evaluated and kept. They are evaluated this once only: a variable they were worked out from can change inside the loop without changing them.
2. The test: if *step* is zero or more and *v* is greater than *limit*, or *step* is negative and *v* is less than *limit*, the loop ends, and the program goes on after its `NEXT`. The test is made before the first pass too, so the statements may not run at all; *v* then keeps the value *start*.
3. Otherwise the statements run.
4. At `NEXT`, *step* is added to *v*, and the loop goes back to the test at step 2.

When the loop ends at the test, *v* holds the first value that failed it: after `FOR I = 1 TO 3`, `I` is 4.

The addition is exact. A loop with a fractional step, `STEP 0.1` or `STEP 1/3`, reaches its limit on the last pass, where a binary floating-point BASIC may stop one short.

The statements may assign to *v*; `NEXT` adds *step* to whatever it then holds.

`EXIT FOR` leaves the innermost loop at once, with *v* as it is.

`NEXT` with no variable closes the innermost open loop. `NEXT v` does the same, and the compiler checks that *v* is that loop's counter. `NEXT j, i` is `NEXT j : NEXT i`.

ERRORS

ERR	MESSAGE	CAUSE
26	`FOR without NEXT`	A `FOR` whose `NEXT` isn't there. Found when the program is compiled: nothing has run
1	`NEXT without FOR`	A `NEXT` with no loop open, or naming a variable that isn't the innermost loop's counter
80	`block without its start or end`	The loop and another block overlap: a `NEXT` inside a one-line `IF`, say
13	`type mismatch`	*v* is a string variable, or *start*, *limit* or *step* a string
2	`syntax error`	*v* is not a simple variable

NOTES

With a *step* of 0 the counter never changes and the loop doesn't end by itself: only `EXIT FOR`, an error or CTRL-C ends it.

At the prompt, a line is compiled alone, so a loop typed there must be whole on its line: `FOR I = 1 TO 3` by itself is `FOR without NEXT`.

SEE ALSO

`EXIT`, `DO`...`LOOP`, `WHILE`...`WEND`.

EXAMPLE

```
> FOR I = 1 TO 3 : PRINT I; : NEXT : PRINT "then"; I
 1  2  3 then 4
> FOR I = 5 TO 1 : PRINT I; : NEXT : PRINT "then"; I
then 5
> FOR X = 0 TO 1 STEP 1/3 : PRINT X; : NEXT : PRINT
 0  1/3  2/3  1
> N = 3 : FOR I = 1 TO N : N = 10 : PRINT I; : NEXT : PRINT
 1  2  3
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : PRINT I; J; : NEXT J, I : PRINT
 1  1  1  2  2  1  2  2
> FOR I = 1 TO 2 : FOR J = 1 TO 2 : NEXT I
NEXT without FOR
```

INPUT

A statement: reads a line and assigns its fields to variables. The line comes from the console, or from a file.

SYNTAX

```
INPUT [;] ["prompt" {; | ,}] v [, v] ...
INPUT #f, v [, v] ...
```

ARGUMENTS

prompt

A string in quotes, written before the line is read. It must be a literal: a string variable there is a syntax error. After it, `;` adds a question mark and a space, and `,` adds nothing. With no prompt at all, `?` is written.

v

A numeric or string variable, or an array's element. Each takes one field of the line.

f

The number of a file opened `FOR INPUT` or `FOR BINARY`.

DESCRIPTION

From the console. The prompt is written and one line is read, with the console's line editing (section 13.2). Only its first 255 bytes are used. The line is split into fields at its commas, and each variable takes a field, in order:

- A numeric variable's field is read as a number, in any form a program's own numbers may take: `12`, `-7/2`, `2.5`, `1e3`, `&HFF`. An empty field is 0.
- A string variable's field is the text up to the next comma, with the spaces before and after it dropped. A field that starts with a double quote runs to the closing quote instead, and can hold commas and spaces.

If the line has fewer fields than there are variables, or more, or a numeric field that isn't a number, HyBASIC writes `?Redo from start` and the statement starts again from its prompt. It is never an error.

From a file. One line of file *f* is read, and split and assigned in the same way, with three differences. There is no prompt and nothing is asked again: a numeric field that isn't a number is 0. If the line runs out of fields before the variables do, the next line is read and its fields carry on. Fields left on the line after the last variable is filled are dropped: the next `INPUT #` starts at the next line.

The semicolon after `INPUT` matters only when input doesn't come from the console (`basic prog <answers`). The console echoes the Return that ends a line; a file doesn't, so HyBASIC writes a new line itself after reading one. `INPUT ;` leaves it out, and what is printed next follows the prompt on the same line.

ERRORS

ERR	MESSAGE	CAUSE
62	`input past end of file`	No line is left: in file *f*, or in standard input when it isn't the console
52	`bad file number`	File *f* isn't open
54	`bad file mode`	File *f* is open `FOR OUTPUT` or `FOR APPEND`
2	`syntax error`	The prompt isn't a string literal

CTRL-C at the prompt stops the program, as it does anywhere else (`Break`).

NOTES

To read a whole line, commas and all, into one string, use `LINE INPUT`. To read keys without waiting for Return, use `INKEY$` or `INPUT$`.

SEE ALSO

`LINE INPUT`, `INPUT$`, `INKEY$`, `OPEN`, `EOF`, `READ`.

EXAMPLE

```
> 10 INPUT "Two numbers"; A, B
> 20 PRINT A + B
> 30 INPUT "Name, age: ", N$, G
> 40 PRINT "["; N$; "]"; G
> 50 INPUT X, Y$
> 60 PRINT X; "["; Y$; "]"
> RUN
Two numbers? 3
?Redo from start
Two numbers? 1/2, 2.5
 3
Name, age: Sam Smith , 12
[Sam Smith] 12
? , "x, y"
 0 [x, y]
```

Fields that carry on from one line of a file to the next:

```
> 10 OPEN "/ram/n.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "1" : PRINT #1, "2, 3" : PRINT #1, "4"
> 30 CLOSE #1 : OPEN "/ram/n.txt" FOR INPUT AS #1
> 40 INPUT #1, A, B : PRINT A; B
> 50 INPUT #1, C : PRINT C
> 60 INPUT #1, D
> RUN
 1  2
 4
line 60: input past end of file
```

OPEN

A statement: opens a file, and gives it a number for the statements that read and write it.

SYNTAX

```
OPEN path$ FOR {INPUT | OUTPUT | APPEND | BINARY} AS [#]f [LEN = n]
```

ARGUMENTS

path$

A string expression: the file's name. A name that doesn't start with `/` is taken from the current directory (`CHDIR`). It can be any name in the program's namespace: a file on a disk, or a device such as `/dev/cons`.

f

A numeric expression from 1 to 255: the number the file is known by until it is closed. The `#` is optional.

n

Accepted, evaluated and ignored. QuickBASIC's record length has no meaning here: `GET` and `PUT` take a position in bytes.

DESCRIPTION

The mode decides how the file is opened, and which statements may then use it:

MODE	A FILE THAT EXISTS	A FILE THAT DOESN'T	STARTS AT	MAY THEN BE USED BY
`INPUT`	Opened for reading	Error 53	Its first byte	`INPUT #`, `LINE INPUT #`, `INPUT$`, `EOF`, `LOF`, `LOC`, `SEEK`
`OUTPUT`	Emptied, and opened for writing	Made	Its first byte	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`APPEND`	Opened for writing, its contents kept	Made	Its end	`PRINT #`, `WRITE #`, `LOF`, `LOC`, `SEEK`
`BINARY`	Opened for reading and writing	Made	Its first byte	All of the above, and `GET` and `PUT`

Up to 8 files can be open at once. A file stays open until `CLOSE` names it, `CLOSE` with no number closes them all, or a `RUN` starts (which closes every file first). Leaving HyBASIC closes them too.

`EOF(f)` is always true (–1) for a file opened `FOR OUTPUT` or `FOR APPEND`.

ERRORS

ERR	MESSAGE	CAUSE
52	`bad file number`	*f* is less than 1 or more than 255
55	`file already open`	Number *f* is in use
67	`too many files`	8 files are open already
53	`file not found`	`FOR INPUT`: there's no such file. Any mode: a directory in the path isn't there
70	`permission denied`	The file, or its disk (`/rom`), can't be written
13	`type mismatch`	*path$* is a number, or *f* a string

The system's other refusals come through as its own errors (a full disk is 61, `disk full`).

NOTES

The limit of 8 is HyBASIC's own. Each open file also takes one of the task's 16 file descriptors, three of which are the console's.

One file can be open under two numbers at once, each with its own position: opening it `FOR INPUT` as `#2` while it is open `FOR OUTPUT` as `#1` is allowed, and reads what has been written so far.

A file written with `PRINT #` holds exactly the bytes `PRINT` would have sent to the screen, each line ended by a line feed (byte 10).

SEE ALSO

`CLOSE`, `PRINT`, `WRITE`, `INPUT`, `LINE INPUT`, `GET`, `PUT`, `SEEK`, `EOF`, `LOF`, `LOC`, `KILL`, `NAME`.

EXAMPLE

```
> OPEN "/ram/log.txt" FOR OUTPUT AS #1 : PRINT #1, "one" : PRINT LOF(1); EOF(1) : CLOSE
#1
 4 -1
> OPEN "/ram/log.txt" FOR APPEND AS #1 : PRINT #1, "two" : PRINT LOF(1) : CLOSE #1
 8
> OPEN "/ram/log.txt" FOR INPUT AS #1 : LINE INPUT #1, L$ : PRINT L$; EOF(1)
one 0
> PRINT #1, "three"
bad file mode
> OPEN "/ram/other.txt" FOR OUTPUT AS #1
file already open
> CLOSE : OPEN "/ram/nothere.txt" FOR INPUT AS #1
file not found
> PRINT ERR
 53
```

CHAPTER 2

Commands

A command is a program in `/bin`, or one of the shell's own. Each ends with an **exit status**, which rc keeps in `$status`: empty for success, and a short text for failure. Most commands fail with the status `1`, and with `usage` when their arguments make no sense. A command's messages go to standard error, as `name: what: why`, so they aren't mixed into a pipe's data.

A path in an argument can be at most 63 characters long once it has been made absolute.

cp

A program (`/bin/cp`): copies files, and with `-r` directories.

SYNTAX

```
cp [-r] from to
cp [-r] from ... dir
```

ARGUMENTS

-r

Copy directories too: each one's whole tree.

from

A file to copy, or with `-r` a directory. There can be several if the last name is a directory.

to

The copy's name. If it is an existing directory, the copy goes into it under *from*'s own name.

dir

An existing directory. Each *from* is copied into it under its own name.

DESCRIPTION

Each *from* is handled in turn, and one that fails doesn't stop the rest.

A file is copied by opening it for reading, creating the destination, and reading and writing 512 bytes at a time until a read returns nothing. Creating the destination empties a file that is already there. The copy is a new file: it has its own date, the time of copying, and is not read-only even if the original is.

Before copying, `cp` compares the two files' device, disk and file identifier. If they are the same file, by whatever two names, it refuses to copy it onto itself, which would otherwise empty it.

Anything that can be opened and read can be a *from*, a device included: `cp /dev/time now` copies what one read of the clock gives.

Without `-r`, a *from* that is a directory is reported and skipped. With `-r`, the directory is made at the destination (one already there is used as it is), and then everything in it is copied in the same way, down to 16 directories deep.

EXIT STATUS

Empty if every file was copied. `1` if any was not. `usage`, after a line saying how `cp` is used, if there are fewer than two names or an option other than `-r`.

ERRORS

Each is printed as `cp: name: reason`, where *name* is the file that could not be read or written.

REASON	CAUSE
`not found`	*from* isn't there, or a directory in *to*'s path isn't
`is a directory`	*from* is a directory, and there's no `-r`
`not a directory`	Several *from*s, and the last name isn't a directory
`the same file`	*from* and *to* are one file
`read-only`	The destination's disk can't be written (`/rom`)
`name too long`	The destination's path would pass 63 characters

Any other reason is the system's own error message for what failed (Appendix G): a full disk, for one.

NOTES

If a read or a write fails part-way, the destination is left holding what was written up to then.

`cp` doesn't ask before it overwrites.

SEE ALSO

`mv`, `rm`, `mkdir`, `cat`, `cmp`.

EXAMPLE

```
% echo hello >a.txt
% cp a.txt b.txt; cat b.txt
hello
% mkdir d; cp a.txt b.txt d; ls d
a.txt
b.txt
% cp d e; echo $status
cp: d: is a directory
1
% cp -r d e; ls e
a.txt
b.txt
% cp a.txt a.txt
cp: a.txt: the same file
% cp a.txt /rom/x
cp: /rom/x: read-only
% cp a.txt; echo $status
usage: cp [-r] from to, or cp [-r] from ... dir
usage
```

CHAPTER 3

System Calls

A system call is a `jsr` to a fixed address in the jump table at `$F800`. Its arguments go in A, X, Y and the call registers r0 to r15 (two bytes each, in the zero page from `$02`); its results come back in the same places, a 16-bit result in A (low byte) and X (high byte). On return, C = 0 means success. C = 1 means failure, and A holds an error code (Appendix G). A call may change A, X, Y, the flags and r0 to r15, including registers it doesn't use for a result; it never changes the program's own zero page, `$22` to `$7F`.

A call that *can wait* may suspend the task until something happens elsewhere: a key, a disk, another task. A note delivered to the task ends such a wait, and the call returns `E_INTR`.

Each call's name is defined in `hydra.inc` (`/lib/as`), with the constants and error codes used here.

OPEN

System call `$F8C0`: opens a file that exists, and returns a file descriptor for it. It can wait.

SYNTAX

```
            LDR r0, path        ; r0 = the address of the path
            lda #mode
            jsr OPEN
            bcs failed          ; A = the error code
            sta fd              ; A = the file descriptor
```

ARGUMENTS

r0

The address of the path: its bytes, then a zero byte.

A

The mode: one of `O_READ` (0), `O_WRITE` (1) or `O_RDWR` (2), to which can be added `O_TRUNC` (`$10`), to empty the file as it is opened, and `O_NONBLOCK` (`$40`), to make reads and writes of this descriptor return `E_AGAIN` where they would otherwise wait.

RESULT

C = 0, and A = the file descriptor: a number from 0 to 15, the lowest one the task has free.

DESCRIPTION

1. The path is made absolute and clean. One that doesn't start with `/` (or with `#`, a device's own name) has the task's current directory put before it; then `.` elements and doubled slashes are removed, and each `..` removes the element before it. The result may be at most 63 bytes (`PATH_MAX`).
2. The path is looked up in the task's namespace (section 9.5). Where a directory is a union of several (section 9.7), each member is tried in turn, and the first that has the name is the one opened. A member whose device has gone is passed over as if it didn't have the name.
3. The device's driver is asked to open the file in the mode given. If it refuses, its error is the call's.
4. The descriptor is set to the file, at offset 0.

A descriptor has an offset of its own, which `READ`, `WRITE` and `SEEK` move. Descriptors made from it by `DUP`, and those a child task is given by `SPAWN`, share that offset; a second `OPEN` of the same file does not.

Opening a directory with `O_READ` succeeds: it reads as a sequence of 64-byte stat records, one for each file in it. When the directory is a union, the records of all its members follow one another.

`OPEN` never makes a file. `CREATE` does, and opens it.

ERRORS

CODE	NAME	CAUSE
`$20`	`E_NOENT`	No such file, in any member of a union
`$2B`	`E_NODEV`	The path names a device that isn't there
`$0A`	`E_NAMETOOLONG`	The path, made absolute, is longer than 63 bytes
`$26`	`E_MFILE`	The task has all 16 of its descriptors in use
`$27`	`E_NFILE`	All 48 of the system's channels are in use (every open file in every task takes one; `DUP`s share)
`$05`	`E_INTR`	A note arrived while the call was waiting for the driver

A driver adds its own: `E_ROFS` (`$29`) for a write mode on a read-only disk, and `E_ISDIR` (`$23`) for a write mode on a directory.

NOTES

Descriptors 0, 1 and 2 are standard input, output and error, already open when a program starts. A task that closes one of them will get that number from its next `OPEN`.

A descriptor that isn't closed is closed when the task ends.

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-open` (*z-addr mode -- fd ior*), the path zero-terminated (`>z` makes one)
hylang	`(sys-open path [mode])`: the descriptor. A failure is an error, its text the path and the system's message
HyBASIC	`SYS "OPEN"`; or, for most purposes, the `OPEN` statement (Chapter 1)

SEE ALSO

`CREATE`, `CLOSE`, `READ`, `WRITE`, `SEEK`, `DUP`, `FSTAT`.

EXAMPLE

From hylang. Descriptors 0 to 2 are in use, so the first file opened is 3:

```
hylang> (sys-open "/dev/time" 0)
=> 3
hylang> (sys-open "/dev/nothing" 0)
=> Error: /dev/nothing: not found
hylang> (sys-open "/rom/bin/ls" 1)
=> Error: /rom/bin/ls: read-only
hylang> (sys-open "/ram" 1)
=> Error: /ram: is a directory
hylang> (sys-open "#q/x" 0)
=> Error: #q/x: no such device
```

READ

System call `$F8C9`: reads bytes from an open file into memory. It can wait.

SYNTAX

```
            LDR r0, buf         ; r0 = where the bytes go
            LDR r1, count       ; r1 = how many, at most
            lda fd
            jsr READ
            bcs failed          ; A = the error code
                                ; A/X = how many were read; 0 is the end of the file
```

ARGUMENTS

A

A file descriptor, open with `O_READ` or `O_RDWR`.

r0

The address of a buffer in the task's memory.

r1

The most bytes to read: 0 to 65,535. The buffer must have room for them.

RESULT

C = 0, and A/X = the number of bytes read, which were put in the buffer from its start. The descriptor's offset has moved on by that many.

DESCRIPTION

The kernel asks the file's driver for the bytes at the descriptor's offset, at most 512 at a time (`IO_UNIT`), and repeats until one of three things happens:

- **The count is reached.** The result equals r1.
- **The file ends.** The result is what there was before the end; a read that starts at the end returns 0. A result of 0 is how the end of a file is recognised, and it is not an error.
- **The driver returns fewer bytes than it was asked for.** The kernel stops there and returns them. This is how a device says "that is all for now": the console returns one line, however large the buffer.

A result smaller than r1 therefore does not by itself mean the end of the file. A program that needs exactly *n* bytes must call `READ` again for the remainder until it has them or gets 0.

When the driver has nothing yet (no key typed, an empty pipe), the task waits. With `O_NONBLOCK` set on the descriptor, the call returns `E_AGAIN` instead.

A directory reads as 64-byte stat records (`SR_SIZE`), one for each file in it: ask for a multiple of 64.

ERRORS

CODE	NAME	CAUSE
`$25`	`E_BADF`	A isn't an open descriptor, or it was opened with `O_WRITE`
`$04`	`E_AGAIN`	Nothing to read yet, and the descriptor is non-blocking
`$05`	`E_INTR`	A note arrived while the call was waiting

A driver adds its own: a disk that fails answers `E_IO` (`$2A`).

FROM THE LANGUAGES

LANGUAGE	FORM
HyForth (`lib hydra`)	`sys-read` (*fd c-addr u -- u2 ior*): *u2* bytes were read
hylang	`(sys-read fd count)` : the bytes read, as a string
HyBASIC	`SYS "READ"`; or `INPUT #`, `LINE INPUT #`, `INPUT$` and `GET`

SEE ALSO

`OPEN`, `WRITE`, `SEEK`, `CLOSE`, `PIPE`.

EXAMPLE

The offset moving on, a count cut short by the end of the file, and 0 at the end:

```
hylang> (set {fd} (sys-open "/ram/t" 0))
=> NIL
hylang> (sys-read fd 5)
=> "hello"
hylang> (sys-read fd 100)
=> " world\n"
hylang> (sys-read fd 100)
=> ""
hylang> (sys-close fd)
=> NIL
```

CHAPTER 4

HyForth Words

A cell is 16 bits. A double is two cells, 32 bits, its high cell on top of the stack. A flag is 0 for false and –1 (all bits set) for true. *ior* is 0 for success and a non-zero error code otherwise. Stack pictures follow the Forth standard's notation: *n* signed, *u* unsigned, *d* and *ud* doubles, *c-addr u* the address and length of a string.

Each entry says which word set a word belongs to, and whether it is in the ROM core or needs a library loaded with `lib` first. Where a word is the standard's, the entry describes what HyForth does in the cases the standard leaves open.

>number

Core word set, in the ROM core. Converts the digits at the start of a string, adding them to a double.

SYNTAX

```
>number  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

ARGUMENTS

ud1

The number so far: 0 to start a new one.

c-addr1 u1

The text to convert.

RESULT

ud2

ud1 with each converted digit added: for every digit, *ud* × `base` + the digit's value.

c-addr2 u2

What was not converted: the address of the first character that isn't a digit and the number of characters from there to the end. *u2* is 0 if the whole string was digits, and *c-addr2* is then the address just past it.

DESCRIPTION

Characters are taken from the left for as long as each is a digit in the current `base`. The digits are `0` to `9`, then the letters `A` to `Z` for 10 to 35; small letters count as capitals. A character whose value is not less than `base` is not a digit, and conversion stops at it: in decimal `a` stops it, in hex `g` does.

Nothing else is understood. A sign, a prefix (`$`, `#` or `%`) and a decimal point all stop the conversion like any other non-digit; an empty string converts nothing and returns *ud1* unchanged. Signs and prefixes are the text interpreter's business: it deals with them itself, and calls this word's code for the digits.

ERRORS

None. A string with no digits is not an error: *ud2* = *ud1* and *u2* = *u1*.

NOTES

A value that passes 32 bits wraps: only its low 32 bits are kept, and no error is raised. (The standard calls this an ambiguous condition.) To read a number of any length, use the numbers library (`lib numbers`).

`base` can be 2 to 36.

SEE ALSO

`base`, `decimal`, `hex`, `s>d`, `evaluate`.

EXAMPLE

`nip` drops the address, leaving the double (its low cell first, as `.s` shows it) and the count of characters not converted:

```
/> 0 0 s" 123abc" >number nip .s
<3> 123 0 3
/> drop 2drop
/> hex 0 0 s" ff zz" >number nip .s decimal
<3> FF 0 3
/> drop 2drop
/> 0 0 s" -5" >number nip .s
<3> 0 0 2
/> drop 2drop
/> 0 0 s" 99999999999" >number nip .s
<3> 2839 -20518 0
```

CHAPTER 5

hylang Forms

An expression in parentheses is evaluated: its first element names a function or a special form, and the rest are its arguments. A function's arguments are evaluated before it is called, left to right. A **special form** receives its arguments as they were written, and decides for itself which to evaluate and when; each entry for one says exactly that. An expression in braces, a Q-expression, is data: it is not evaluated until something runs it.

A name is looked up in the current **scope**, then in the scope that encloses it, and so on out to the global scope. Each call of a function has a scope of its own, enclosed by the scope the function was made in.

An error is a value: `Error:` and a message. An expression that is given an error as an argument returns it at once, so an error passes outward through whatever contains it until something catches it or it reaches the prompt.

`let`

A special form: evaluates expressions in a new scope, with names bound in it first.

SYNTAX

```
(let {{name [value]} ...} body ...)
(let body)
```

ARGUMENTS

name

A symbol.

value

An expression: what the name is bound to. If it is left out, the name is bound to `NIL`.

body

An expression. It can also be written as a Q-expression, `{print x}`, which is then run as code.

RESULT

The value of the last *body* expression.

DESCRIPTION

1. A new scope is made, enclosed by the scope the `let` is in.

2. The bindings are taken in order. For each, *value* is evaluated **in the new scope**, and *name* is bound to the result there. A value can therefore use the names bound before it in the same `let`; a name bound twice keeps its last value.
3. The *body* expressions are evaluated in order, in the new scope. The values of all but the last are discarded.
4. The new scope is dropped, unless something made in it still refers to it: a function made by `fn` in the body keeps the scope, and can go on using its names after the `let` has returned (a closure).

A name bound by `let` hides one of the same name outside it for as long as the body runs, and the outer one is unchanged afterwards.

Inside the body, `set` binds or changes a name in the `let`'s own scope; `set!` changes a name in whichever scope it was made, which may be outside; and `def` always binds globally.

The last body expression is evaluated in tail position: a function whose body ends in a `let` that ends in a call to itself runs in constant stack, however many times it repeats.

With one argument only, that argument is the body: `(let body)` evaluates it in a new, empty scope. This means `(let {{a 1}})` does not bind anything; its one argument is run as code, and the result is the list `{a 1}`.

ERRORS

MESSAGE	CAUSE
`A 'let' binding is {name value}`	A binding that isn't a list, whose first element isn't a symbol, or that has more than two elements
`'let' supplied too few parameters`	No arguments at all

An error from evaluating a *value* or a *body* expression ends the `let` at once, and is its result.

SEE ALSO

`def`, `set`, `set!`, `fn`, `do`.

EXAMPLE

```
hylang> (let {{x 2} {y (+ x 1)}} (list x y))
=> {2 3}
hylang> (def {x} 10)
=> NIL
hylang> (let {{x 1}} (set {x} 5) x)
=> 5
hylang> x
=> 10
hylang> (def {add5} (let {{n 5}} (fn {a} {+ a n})))
=> NIL
hylang> (add5 1)
=> 6
hylang> n
=> Error: Unbound Symbol 'n'
hylang> (let {a 1} a)
=> Error: A 'let' binding is {name value}
```